

LAPORAN PRATIKUM PEKAN 8 STRUKTUR DATA

“Tree & Graph Traversal”



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Avero

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum Mata Kuliah Praktikum Struktur Data, dengan judul: “Tree & Graph Traversa”. Laporan ini berisikan hasil analisis dari praktikum basis data penulis di kelas DIF62111/IF/Prakt/C yang dilaksanakan pada hari Selasa, tanggal 10 Juni 2026.

Pada kesempatan ini, Penulis ingin mengucapkan terima kasih kepada Bapak Ir Wahyudi, S.T, M.T, dosen pengampu Mata Kuliah Praktikum Struktur Data, yang telah memberikan tugas ini. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis sadar bahwa laporan ini masih memiliki beberapa kekurangan dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis harap adanya bentuk saran dan kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi perkembangan dunia pendidikan.

Padang, 12 Mei 2026

Hamdi Sidqi Alifi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat Pratikum.....	3
BAB II PEMBAHASAN	4
2.1 Node	4
2.2 GraphTraversal.....	6
2.3 BTree.....	8
2.4 BTreeDriver	10
BAB III KESIMPULAN	12
3.1 Kesimpulan	12
3.2 Saran.....	12
BAB IV DAFTAR PUSTAKA	13
Lampiran Tugas	15
1. Deskripsi Tugas.....	15
2. Aturan Penamaan	15
3. Struktur Program.....	15
GRAPH SEARCH GUI	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data pohon (tree) dan graf (graph) merupakan dua struktur data non-linear yang sangat penting dalam ilmu komputer. Pohon digunakan untuk merepresentasikan hierarki data, seperti sistem file, pohon ekspresi, dan pohon keputusan. Sementara itu, graf digunakan untuk merepresentasikan relasi antara entitas yang lebih kompleks, seperti jaringan sosial, peta jalan, dan jaringan komputer.

Pada pertemuan ini, kita mempelajari implementasi Binary Tree (pohon biner) beserta operasi traversal-nya (Inorder, Preorder, Postorder) serta implementasi Graph dengan dua metode penelusuran utama, yaitu Depth First Search (DFS) dan Breadth First Search (BFS). Pemahaman terhadap kedua struktur data ini sangat krusial untuk membangun fondasi yang kuat dalam pemrograman dan algoritma.

1.2 Tujuan

1. Memahami implementasi Binary Tree dalam bahasa pemrograman Java.
2. Memahami operasi traversal pada Binary Tree (Inorder, Preorder, Postorder).
3. Memahami implementasi Graf (Graph) menggunakan Adjacency List.
4. Memahami algoritma DFS dan BFS pada Graf.

1.3 Manfaat Pratikum

1. Mampu mengimplementasikan struktur data Binary Tree dalam Java.
2. Mampu melakukan traversal pohon biner dengan metode Inorder, Preorder, dan Postorder.
3. Mampu mengimplementasikan struktur data Graf menggunakan HashMap.
4. Mampu menelusuri graf menggunakan algoritma DFS dan BFS

BAB II PEMBAHASAN

2.1 Node

Kelas `Node_2511531009` merepresentasikan sebuah simpul (node) dalam struktur Binary Tree. Setiap node menyimpan data integer dan referensi ke anak kiri serta anak kanan. Kelas ini juga mengandung logika traversal pohon secara rekursif.

```
1 package pekan9_2511531009;
2
3 public class Node_2511531009 {
4     int data_1009;
5     Node_2511531009 left_1009;
6     Node_2511531009 right_1009;
7     public Node_2511531009 (int data_1009) {
8         this.data_1009 = data_1009;
9         left_1009 = null;
10        right_1009 = null;
11    }
12    public void setLeft_1009 (Node_2511531009 node_1009) {
13        if (left_1009 == null)
14            left_1009 = node_1009;
15    }
16    public void setRight_1009 (Node_2511531009 node_1009) {
17        if (right_1009 == null)
18            right_1009 = node_1009;
19    }
20    public Node_2511531009 getLeft_1009 () {
21        return left_1009;
22    }
23    public Node_2511531009 getRight_1009 () {
24        return right_1009;
25    }
26    public int getData_1009 () {
27        return data_1009;
28    }
29    public void setData_1009 (int data_1009) {
30        this.data_1009 = data_1009;
31    }
32 }
```

Constructor: `Node_2511531009(int data_1009)` Terdapat pada baris 7 hingga 11. Constructor ini menginisialisasi sebuah node baru dengan nilai data yang diberikan. Referensi `left_1009` dan `right_1009` diinisialisasi dengan null, menandakan node belum memiliki anak.

`setLeft_1009(Node_2511531009 node_1009)` pada baris 12 hingga 15. Method ini menetapkan anak kiri dari node saat ini. Pengecekan kondisi `if (left_1009 == null)` memastikan anak kiri hanya dapat diset satu kali — yakni hanya jika belum memiliki anak kiri sebelumnya.

`setRight_1009(Node_2511531009 node_1009)` pada baris 16 hingga 19. Identik dengan `setLeft_1009`, tetapi beroperasi pada anak kanan (`right_1009`). Hanya menetapkan nilai jika `right_1009` masih null.

`getLeft_1009()` / `getRight_1009()` pada baris 20 hingga 25. Dua getter ini mengembalikan referensi ke anak kiri dan kanan dari node secara berturut-turut. Digunakan oleh kelas `BTree_2511531009` untuk menelusuri pohon

`getData_1009()` / `setData_1009(int data_1009)` pada baris 26 hingga 32. `getData_1009()` mengembalikan nilai integer yang tersimpan di node,

sementara setData_1009() memperbarui nilai tersebut. Keduanya merupakan accessor/mutator standar.

```
32 void printPreorder_1009 (Node_2511531009 node_1009) {
33     if (node_1009 == null)
34         return;
35     System.out.print(node_1009.data_1009 + " ");
36     printPreorder_1009 (node_1009.left_1009);
37     printPreorder_1009 (node_1009.right_1009);
38 }
39 void printPostorder_1009 (Node_2511531009 node_1009) {
40     if (node_1009 == null)
41         return;
42     printPostorder_1009 (node_1009.left_1009);
43     printPostorder_1009 (node_1009.right_1009);
44     System.out.print(node_1009.data_1009 + " ");
45 }
46 void printInorder_1009 (Node_2511531009 node) {
47     if (node == null)
48         return;
49     printInorder_1009 (node.left_1009);
50     System.out.print(node.data_1009 + " ");
51     printInorder_1009 (node.right_1009);
52 }
53 }
```

printPreorder_1009(Node_2511531009 node_1009) pada baris 33–39. Melakukan traversal Preorder secara rekursif: cetak data node saat ini terlebih dahulu, kemudian telusuri anak kiri, lalu anak kanan (Root → Left → Right). Base case: kembalikan jika node == null.

printPostorder_1009(Node_2511531009 node_1009) pada baris 40–46. Melakukan traversal Postorder: telusuri anak kiri terlebih dahulu, lalu anak kanan, kemudian cetak data node saat ini (Left → Right → Root). Berguna untuk menghapus pohon atau mengevaluasi ekspresi.

printInorder_1009(Node_2511531009 node) pada baris 47–53. Melakukan traversal Inorder: telusuri anak kiri, cetak data node saat ini, lalu telusuri anak kanan (Left → Root → Right). Pada BST, traversal ini menghasilkan urutan data yang terurut secara ascending.

```
54 public String print_1009 () {
55     return this.print_1009("", true, "");
56 }
57 public String print_1009 (String prefix_1009, boolean isTail_1009, String sb_1009) {
58     if (right_1009 != null) {
59         right_1009.print_1009(prefix_1009 + (isTail_1009 ? "| " : " "), false, sb_1009);
60     }
61     System.out.println(prefix_1009 + (isTail_1009 ? "\\--" : "/--") + data_1009);
62     if (left_1009 != null) {
63         left_1009.print_1009(prefix_1009 + (isTail_1009 ? " " : "| "), true, sb_1009);
64     }
65     return sb_1009;
66 }
67 }
68 }
```

print_1009() / print_1009(String prefix_1009, boolean isTail_1009, String sb_1009) Method ini mencetak representasi visual pohon ke konsol menggunakan karakter ASCII (\\--, /--, |). Metode overloaded pertama memanggil versi kedua dengan parameter awal. Rekursi dimulai dari anak kanan terlebih dahulu (sehingga pohon tampak dengan root di kiri), kemudian cetak node saat ini, lalu anak kiri.

2.2 GraphTraversal

Kelas GraphTraversal_2511531009 mengimplementasikan struktur data Graf tidak berarah (undirected graph) menggunakan Adjacency List berbasis HashMap. Kelas ini menyediakan metode untuk menambah sisi, mencetak graf, dan menelusuri graf menggunakan DFS serta BFS.

```
1 package pekan9_2511531009;
2 import java.util.*;
3 public class GraphTraversal_2511531009 {
4     private Map<String, List<String>> graph_1009 = new HashMap<>();
5
6     public void addEdge_1009(String node1_1009, String node2_1009) {
7         graph_1009.putIfAbsent(node1_1009, new ArrayList<>());
8         graph_1009.putIfAbsent(node2_1009, new ArrayList<>());
9         graph_1009.get(node1_1009).add(node2_1009);
10        graph_1009.get(node2_1009).add(node1_1009);
11    }
12
13    public void printGraph_1009() {
14        System.out.println("Graf Awal (Adjacency List): ");
15        for (String node_1009 : graph_1009.keySet()) {
16            List<String> neighbors_1009 = graph_1009.get(node_1009);
17            System.out.println(node_1009 + " -> " + String.join(", ", neighbors_1009));
18        }
19    }
20
21    public void dfs_1009(String start_1009) {
22        Set<String> visited_1009 = new HashSet<>();
23        System.out.println("Penelusuran DFS");
24        dfsHelper_1009(start_1009, visited_1009);
25        System.out.println();
26    }
27
28    private void dfsHelper_1009(String current_1009, Set<String> visited_1009) {
29        if (visited_1009.contains(current_1009)) {
30            return;
31        }
32        visited_1009.add(current_1009);
33        System.out.print(current_1009 + " ");
34        for (String neighbor_1009 : graph_1009.getOrDefault(current_1009, new ArrayList<>())) {
35            dfsHelper_1009(neighbor_1009, visited_1009);
36        }
37    }
```

graph_1009 pada aris 4. Sebuah Map> yang merupakan representasi Adjacency List dari graf. Setiap kunci (key) adalah nama node, dan nilainya (value) adalah daftar node tetangga yang terhubung langsung.

addEdge_1009(String node1_1009, String node2_1009) pada baris 6–11. Menambahkan sisi (edge) antara dua node. Method putIfAbsent memastikan setiap node memiliki entri dalam map. Karena grafnya tidak berarah, sisi ditambahkan dua arah: node1 → node2 dan node2 → node1.

printGraph_1009() pada baris 13–18. Mencetak seluruh isi Adjacency List ke konsol. Iterasi dilakukan pada setiap kunci dalam graph_1009, lalu menampilkan daftar tetangga yang dipisahkan koma menggunakan String.join().

dfsHelper_1009(String current_1009, Set visited_1009) pada baris 26–33. Inti dari algoritma DFS. Base case: jika node sudah ada di visited_1009, method langsung kembali untuk menghindari siklus. Jika belum, node ditambahkan ke visited_1009, dicetak, lalu DFS dilanjutkan secara rekursif ke setiap tetangganya. Pendekatan ini menggunakan stack implisit dari call stack rekursi.

```

39 public void bfs_1009(String start) {
40     Set<String> visited_1009 = new HashSet<>();
41     Queue<String> queue_1009 = new LinkedList<>();
42     queue_1009.add(start);
43     visited_1009.add(start);
44     System.out.println("Penelusuran BFS:");
45     while (!queue_1009.isEmpty()) {
46         String current = queue_1009.poll();
47         System.out.print(current + " ");
48         for (String neighbor : graph_1009.getDefault(current, new ArrayList<>())) {
49             if (!visited_1009.contains(neighbor)) {
50                 queue_1009.add(neighbor);
51                 visited_1009.add(neighbor);
52             }
53         }
54     }
55     System.out.println();
56 }
57
58 public static void main(String[] args) {
59     GraphTraversal_2511531009 graph_1009 = new GraphTraversal_2511531009();
60
61     graph_1009.addEdge_1009("A", "B");
62     graph_1009.addEdge_1009("A", "C");
63     graph_1009.addEdge_1009("B", "D");
64     graph_1009.addEdge_1009("B", "E");
65     System.out.println("Graf Awal adalah: ");
66     graph_1009.printGraph_1009();
67     graph_1009.dfs_1009("A");
68     graph_1009.bfs_1009("A");
69 }
70 }

```

bfs_1009(String start) pada baris 35–48. Mengimplementasikan algoritma BFS menggunakan Queue (LinkedList). Node awal ditambahkan ke antrian dan di-mark sebagai dikunjungi. Selama antrian tidak kosong, node di-dequeue (poll()), dicetak, dan setiap tetangganya yang belum dikunjungi di-enqueue (add()). BFS menjamin urutan kunjungan berdasarkan jarak dari node awal.

main(String[] args) pada baris 50–59. Method utama yang mendemonstrasikan penggunaan kelas ini. Membuat sebuah graf dengan 5 node (A–E) dan 4 sisi: A-B, A-C, B-D, B-E. Kemudian mencetak Adjacency List, menjalankan DFS mulai dari A, dan menjalankan BFS mulai dari A

2.3 BTree

Kelas BTree_2511531009 merepresentasikan struktur data Binary Tree secara keseluruhan. Kelas ini mengelola referensi ke root, menyediakan operasi pencarian (search), penghitungan node, traversal, dan pencetakan visual pohon dengan mendelegasikan logika rekursif ke kelas Node_2511531009.

```
1 package pekan9_2511531009;
2
3 public class BTree_2511531009 {
4     private Node_2511531009 root_1009;
5     private Node_2511531009 currentNode_1009;
6     public BTree_2511531009 () {
7         root_1009 = null;
8     }
9     public boolean search_1009 (int data_1009) {
10         return search_1009 (root_1009, data_1009);
11     }
12     private boolean search_1009 (Node_2511531009 node_1009, int data_1009) {
13         if (node_1009 == null)
14             return false;
15         if (node_1009.getData_1009() == data_1009)
16             return true;
17         if (search_1009(node_1009.getLeft_1009(), data_1009))
18             return true;
19         return search_1009(node_1009.getRight_1009(), data_1009);
20     }
21     public void printInorder_1009() {
22         if (root_1009 != null)
23             root_1009.printInorder_1009(root_1009);
24     }
25     public void printPreorder_1009() {
26         if (root_1009 != null)
27             root_1009.printPreorder_1009(root_1009);
28     }
29     public void printPostorder_1009() {
30         if (root_1009 != null)
31             root_1009.printPostorder_1009(root_1009);
32     }
33 }
```

Atribut: root_1009 & currentNode_1009 pada baris 3–4. root_1009 adalah referensi ke simpul akar (root) pohon. currentNode_1009 digunakan untuk melacak node terakhir yang dioperasikan, berguna sebagai pointer navigasi.

Constructor: BTree_2511531009() pada baris 5–7. Menginisialisasi pohon kosong dengan menetapkan root_1009 = null.

search_1009(int data_1009) publik pada baris 8–10. Method publik yang menjadi entry point pencarian. Mendelegasikan pencarian ke versi privat rekursif dengan root_1009 sebagai node awal.

search_1009(Node_2511531009 node_1009, int data_1009) versi privat Baris: terdapat pada baris 11–18. Implementasi pencarian rekursif. Base case: kembalikan false jika node null. Kembalikan true jika data ditemukan. Jika tidak, rekursif ke anak kiri; jika masih tidak ditemukan, rekursif ke anak kanan. Kompleksitas waktu $O(n)$ karena ini adalah Binary Tree biasa (bukan BST).

printInorder_1009() / printPreorder_1009() / printPostorder_1009() yang melintang dari baris 19–28. Tiga method traversal publik ini memeriksa

apakah pohon tidak kosong (`root_1009 != null`), lalu mendelegasikan ke method traversal rekursif di kelas `Node_2511531009`.

```
34 public Node_2511531009 getRoot_1009() {
35     return root_1009;
36 }
37
38 public boolean isEmpty_1009() {
39     return root_1009 == null;
40 }
41
42 public int countNodes_1009() {
43     return countNodes_1009 (root_1009);
44 }
45
46 private int countNodes_1009 (Node_2511531009 node_1009) {
47     if (node_1009 == null) {
48         return 0;
49     } else {
50         int count_1009 = 1;
51         count_1009 += countNodes_1009 (node_1009.getLeft_1009());
52         count_1009 += countNodes_1009 (node_1009.getRight_1009());
53         return count_1009;
54     }
55 }
56
57 public void print_1009 () {
58     if (root_1009 != null)
59         root_1009.print_1009();
60 }
61
62 public Node_2511531009 getCurrent_1009() {
63     return currentNode_1009;
64 }
65
66 public void setCurrent (Node_2511531009 node_1009) {
67     this.currentNode_1009 = node_1009;
68 }
69
70 public void setRoot_1009 (Node_2511531009 root_1009) {
71     this.root_1009 = root_1009;
72 }
73
74
75 }
```

`getCurrent_1009()` / `setCurrent(Node_2511531009 node_1009)` pada baris 56–60. Getter dan setter untuk `currentNode_1009`. Memungkinkan driver untuk mengatur dan membaca node yang sedang aktif ditunjuk sebagai node 'saat ini'.

2.4 BTreeDriver

Kelas BTreeDriver_2511531009 adalah kelas driver (main class) yang mendemonstrasikan seluruh fungsionalitas Binary Tree. Kelas ini membangun pohon dengan 9 node, menampilkan informasi pohon, dan menjalankan semua jenis traversal.

```
1 package pekan9_2511531009;
2
3 public class BTreeDriver_2511531009 {
4
5     public static void main(String[] args) {
6
7         BTree_2511531009 tree_1009 = new BTree_2511531009();
8         System.out.print("Jumlah simpul awal pohon : ");
9         System.out.println(tree_1009.countNodes_1009());
10
11         Node_2511531009 root_1009 = new Node_2511531009(1);
12
13         tree_1009.setRoot_1009(root_1009);
14
15         System.out.print("Jumlah simpul jika hanya ada root : ");
16         System.out.println(tree_1009.countNodes_1009());
17
18         Node_2511531009 node2_1009 = new Node_2511531009(2);
19         Node_2511531009 node3_1009 = new Node_2511531009(3);
20         Node_2511531009 node4_1009 = new Node_2511531009(4);
21         Node_2511531009 node5_1009 = new Node_2511531009(5);
22         Node_2511531009 node6_1009 = new Node_2511531009(6);
23         Node_2511531009 node7_1009 = new Node_2511531009(7);
24         Node_2511531009 node8_1009 = new Node_2511531009(8);
25         Node_2511531009 node9_1009 = new Node_2511531009(9);
26
27         root_1009.setLeft_1009(node2_1009);
28         node2_1009.setLeft_1009(node4_1009);
29         node2_1009.setRight_1009(node5_1009);
30         node4_1009.setRight_1009(node8_1009);
31
32         root_1009.setRight_1009(node3_1009);
33
34         node3_1009.setLeft_1009(node6_1009);
35         node3_1009.setRight_1009(node7_1009);
36
37         node6_1009.setLeft_1009(node9_1009);
38
39
40         tree_1009.setCurrent(tree_1009.getRoot_1009());
41         System.out.print("menampilkan simpul terakhir : ");
42         System.out.println(tree_1009.getCurrent_1009().getData_1009());
43         System.out.print("Jumlah simpul setelah simpul 7 ditambahkan : ");
44         System.out.println(tree_1009.countNodes_1009());
45         System.out.print("Inorder : ");
46         tree_1009.printInorder_1009();
47         System.out.println();
48         System.out.print("Preorder : ");
49         tree_1009.printPreorder_1009();
50         System.out.println();
51         System.out.print("Postorder : ");
52         tree_1009.printPostorder_1009();
53         System.out.println();
54         System.out.println("\nMenampilkan simpul dalam bentuk pohon");
55         tree_1009.print_1009();
56
57     }
```

main(String[] args) Inisialisasi & Pohon Kosong terdapat pada baris 6–9. Membuat instance BTree_2511531009 dan langsung mencetak jumlah node. Karena belum ada root, countNodes_1009() mengembalikan 0.

main(String[] args) Penetapan Root Baris: pada baris 11–15. Membuat node root dengan nilai 1, lalu menetapkan sebagai akar pohon melalui setRoot_1009(). Setelah ini, countNodes_1009() mengembalikan 1.

main(String[] args) Pembangunan Pohon Baris: 17–39 Baris 17–39. Membuat 8 node tambahan (node2 hingga node9) dengan nilai 2–9, lalu menyusunnya menjadi sebuah pohon dengan struktur hierarki berikut: node1 (root) memiliki anak kiri node2 dan anak kanan node3; node2 memiliki anak kiri node4 dan anak kanan node5; node4 memiliki anak kanan node8; node3

memiliki anak kiri node6 dan anak kanan node7; node6 memiliki anak kiri node9.

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa Binary Tree dan Graf merupakan dua struktur data non-linear yang sangat fundamental. Binary Tree memungkinkan penyimpanan data secara hierarkis dan dapat ditelusuri dengan tiga metode berbeda (Inorder, Preorder, Postorder) yang masing-masing menghasilkan urutan kunjungan node yang berbeda sesuai kebutuhan. Graf dengan representasi Adjacency List sangat fleksibel dan efisien untuk merepresentasikan relasi antar-entitas, sementara DFS dan BFS memberikan dua strategi penelusuran dengan karakteristik yang berbeda, DFS lebih cocok untuk eksplorasi mendalam sementara BFS cocok untuk mencari jalur terpendek.

3.2 Saran

BAB IV

DAFTAR PUSTAKA

- [1] Binary Tree Traversal [Daring]. Tersedia:
<https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/>. Diakses: 10 Juni 2026.
- [2] Breadth First Search (BFS) [Daring]. Tersedia:
<https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>.
Diakses: 10 Juni 2026.
- [3] Depth First Search (DFS) [Daring]. Tersedia:
<https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>.
Diakses: 10 Juni 2026.
- [4] Graph and its representations [Daring]. Tersedia:
<https://www.geeksforgeeks.org/graph-and-its-representations/>. Diakses: 10 Juni 2026.

LAPORAN TUGAS PEKAN 8 STRUKTUR DATA



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas
2026

Lampiran Tugas

Topik : []

Tema : []

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph. Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa: Gedung Perkuliahan Fakultas Universitas Sekolah Rumah Sakit Mall Tempat Wisata Bandara Atau lokasi lainnya Program harus mampu: Menampilkan graph dalam bentuk visual Menentukan titik awal (Start) Menentukan titik tujuan (Goal) Melakukan pencarian menggunakan BFS Melakukan pencarian menggunakan DFS Menampilkan jalur yang ditemukan Tugas BFS dan DFS 1 Menampilkan urutan simpul yang dikunjungi Urutan node yang dikunjungi Jumlah node yang dieksplorasi

2. Aturan Penamaan

a) Nama Kelas : concat sufiks “_” + NIM

Contoh : sorting_2511531009

b) Nama Variabel dan Method : concat sufiks “_” + NIM mod 10^4

Contoh : dataLagu_1009
shellSort_1009

c) Method Utama

Contoh : quickSort_1009 if choose == quick-sort
shellSort_1009 if choose == shell-sort
mergeSort_1009 if choose == merge-sort

3. Struktur Program

a. Vertex and Edge

Minimal 10 simpul (vertex)

Minimal 15 sisi (edge)

Nama simpul harus merepresentasikan lokasi pada peta yang dibuat

b. Method yang Wajib Ada

1. BFS()

Melakukan pencarian menggunakan algoritma Breadth First Search.

2. DFS()

Melakukan pencarian menggunakan algoritma Depth First Search.

3. displayGraph()

Menampilkan graph pada GUI.

c. `displayPath()`

GRAPH SEARCH GUI

```

1 package pekan9_2511531009;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.*;
6 import java.util.List;
7
8 @SuppressWarnings("serial")
9 public class GraphSearchGUI_2511531009 extends JFrame {
10
11     private final Map<String, List<String>> graph_1009 = new LinkedHashMap<>();
12
13     private List<String> foundPath_1009 = new ArrayList<>();
14     private List<String> visitedOrder_1009 = new ArrayList<>();
15     private Set<String> visitedNodes_1009 = new LinkedHashSet<>();
16
17     private JTextArea graphArea_1009;
18     private JComboBox<String> startBox_1009, goalBox_1009;
19     private JTextArea resultArea_1009;
20
21     private static final String[] NODES_1009 = {
22         "Gerbang Utama", "Gedung Rektorat", "Fakultas Teknik",
23         "Lab Komputer", "Aula", "Perpustakaan",
24         "Kantin", "Lapangan", "Parkiran",
25         "Masjid"
26     };
27     private static final int[][] GRID_POS_1009 = {
28         {0, 1}, {1, 1}, {2, 0}, {3, 0}, {4, 0},
29         {2, 1}, {3, 1}, {4, 1}, {0, 2}, {1, 2}
30     };
31
32     public GraphSearchGUI_2511531009() {
33         buildGraph_1009();
34         buildUI_1009();
35     }
36
37     private void buildGraph_1009() {
38         addEdge_1009("Gerbang Utama", "Gedung Rektorat");
39         addEdge_1009("Gerbang Utama", "Parkiran");
40         addEdge_1009("Gedung Rektorat", "Fakultas Teknik");
41         addEdge_1009("Gedung Rektorat", "Perpustakaan");
42         addEdge_1009("Gedung Rektorat", "Masjid");
43         addEdge_1009("Fakultas Teknik", "Lab Komputer");
44         addEdge_1009("Fakultas Teknik", "Kantin");
45         addEdge_1009("Perpustakaan", "Aula");
46         addEdge_1009("Perpustakaan", "Lab Komputer");
47         addEdge_1009("Lab Komputer", "Kantin");
48         addEdge_1009("Aula", "Lapangan");
49         addEdge_1009("Kantin", "Lapangan");
50         addEdge_1009("Parkiran", "Gedung Rektorat");
51         addEdge_1009("Masjid", "Kantin");
52         addEdge_1009("Lapangan", "Masjid");
53     }
54
55     private void addEdge_1009(String a_1009, String b_1009) {
56         graph_1009.computeIfAbsent(a_1009, k_1009 -> new ArrayList<>()).add(b_1009);
57         graph_1009.computeIfAbsent(b_1009, k_1009 -> new ArrayList<>()).add(a_1009);
58     }
59
60     public List<String> BFS(String start_1009, String goal_1009) {
61         visitedOrder_1009 = new ArrayList<>();
62         Map<String, String> parent_1009 = new LinkedHashMap<>();
63         Queue<String> queue_1009 = new LinkedList<>();
64         Set<String> visited_1009 = new LinkedHashSet<>();
65         queue_1009.add(start_1009); visited_1009.add(start_1009); parent_1009.put(start_1009, null);
66         while (!queue_1009.isEmpty()) {
67             String cur_1009 = queue_1009.poll();
68             visitedOrder_1009.add(cur_1009);
69             if (cur_1009.equals(goal_1009)) return buildPath_1009(parent_1009, goal_1009);
70             for (String nb_1009 : graph_1009.getOrDefault(cur_1009, new ArrayList<>())) {
71                 if (!visited_1009.contains(nb_1009)) {
72                     visited_1009.add(nb_1009); parent_1009.put(nb_1009, cur_1009); queue_1009.add(nb_1009);
73                 }
74             }
75         }
76         return new ArrayList<>();
77     }
78
79     public List<String> DFS(String start_1009, String goal_1009) {
80         visitedOrder_1009 = new ArrayList<>();
81         Map<String, String> parent_1009 = new LinkedHashMap<>();
82         Set<String> visited_1009 = new LinkedHashSet<>();
83         parent_1009.put(start_1009, null);
84         dfsHelper_1009(start_1009, goal_1009, visited_1009, parent_1009);
85         return visited_1009.contains(goal_1009) ? buildPath_1009(parent_1009, goal_1009) : new ArrayList<>();
86     }
87
88     private boolean dfsHelper_1009(String cur_1009, String goal_1009, Set<String> visited_1009, Map<String, String> parent_1009) {
89         visited_1009.add(cur_1009); visitedOrder_1009.add(cur_1009);
90         if (cur_1009.equals(goal_1009)) return true;
91         for (String nb_1009 : graph_1009.getOrDefault(cur_1009, new ArrayList<>())) {
92             if (!visited_1009.contains(nb_1009)) {
93                 parent_1009.put(nb_1009, cur_1009);
94                 if (dfsHelper_1009(nb_1009, goal_1009, visited_1009, parent_1009)) return true;
95             }
96         }
97         return false;
98     }
99 }

```

```

100 private List<String> buildPath_1009(Map<String, String> parent_1009, String goal_1009) {
101     LinkedList<String> path_1009 = new LinkedList<>();
102     for (String n_1009 = goal_1009; n_1009 != null; n_1009 = parent_1009.get(n_1009))
103         path_1009.addFirst(n_1009);
104     return path_1009;
105 }
106
107 public void displayGraph() {
108     int[] innerW_1009 = {15, 16, 16, 13, 10};
109
110     Set<String> hEdge_1009 = edgeSet_1009(
111         "Fakultas Teknik", "Lab Komputer",
112         "Lab Komputer", "Aula",
113         "Gerbang Utama", "Gedung Rektorat",
114         "Gedung Rektorat", "Perpustakaan",
115         "Perpustakaan", "Kantin",
116         "Kantin", "Lapangan"
117     );
118     Set<String> vEdge_1009 = edgeSet_1009(
119         "Gerbang Utama", "Parkiran",
120         "Gedung Rektorat", "Masjid",
121         "Fakultas Teknik", "Perpustakaan",
122         "Lab Komputer", "Kantin",
123         "Aula", "Lapangan"
124     );
125
126     StringBuilder sb_1009 = new StringBuilder();
127
128     for (int row_1009 = 0; row_1009 < 3; row_1009++) {
129         StringBuilder top_1009 = new StringBuilder();
130         StringBuilder mid_1009 = new StringBuilder();
131         StringBuilder bot_1009 = new StringBuilder();
132         StringBuilder vert_1009 = new StringBuilder();
133
134         for (int col_1009 = 0; col_1009 < 5; col_1009++) {
135             String name_1009 = nameAt_1009(row_1009, col_1009);
136             int w_1009 = innerW_1009[col_1009];
137             int cellW_1009 = w_1009 + 4;
138
139             if (name_1009 == null) {
140                 top_1009.append(" ".repeat(cellW_1009));
141                 mid_1009.append(" ".repeat(cellW_1009));
142                 bot_1009.append(" ".repeat(cellW_1009));
143             } else {
144                 String border_1009 = "-".repeat(w_1009 + 2);
145                 String prefix_1009, suffix_1009;
146                 if (foundPath_1009.contains(name_1009)) { prefix_1009 = "##"; suffix_1009 = "##"; }
147                 else if (visitedNodes_1009.contains(name_1009)) { prefix_1009 = ">"; suffix_1009 = "<"; }
148                 else { prefix_1009 = "|"; suffix_1009 = "|"; }
149                 top_1009.append("+").append(border_1009).append("+");
150                 mid_1009.append(prefix_1009).append(" ").append(padCenter_1009(name_1009, w_1009)).append(" ").append(suffix_1009);
151                 bot_1009.append("+").append(border_1009).append("+");
152             }
153
154             if (col_1009 < 4) {
155                 String left_1009 = nameAt_1009(row_1009, col_1009);
156                 String right_1009 = nameAt_1009(row_1009, col_1009 + 1);
157                 boolean conn_1009 = left_1009 != null && right_1009 != null && hasEdge_1009(hEdge_1009, left_1009, right_1009);
158                 String dash_1009 = conn_1009 ? (isConsecutive_1009(left_1009, right_1009) ? "====" : "-.-.-") : " ";
159                 top_1009.append(" ");
160                 mid_1009.append(dash_1009);
161                 bot_1009.append(" ");
162             }
163         }
164         sb_1009.append(top_1009).append("\n").append(mid_1009).append("\n").append(bot_1009).append("\n");
165
166         if (row_1009 < 2) {
167             for (int col_1009 = 0; col_1009 < 5; col_1009++) {
168                 String above_1009 = nameAt_1009(row_1009, col_1009);
169                 String below_1009 = nameAt_1009(row_1009 + 1, col_1009);
170                 int cellW_1009 = innerW_1009[col_1009] + 4;
171                 boolean conn_1009 = above_1009 != null && below_1009 != null && hasEdge_1009(vEdge_1009, above_1009, below_1009);
172                 if (conn_1009) {
173                     int half_1009 = cellW_1009 / 2;
174                     vert_1009.append(" ".repeat(half_1009)).append("|").append(" ".repeat(cellW_1009 - half_1009 - 1));
175                 } else {
176                     vert_1009.append(" ".repeat(cellW_1009));
177                 }
178                 if (col_1009 < 4) vert_1009.append(" ");
179             }
180             sb_1009.append(vert_1009).append("\n");
181         }
182     }
183
184     sb_1009.append("\n(~) diagonal/long edges:\n");
185     String[][] diag_1009 = {
186         {"Gedung Rektorat", "Fakultas Teknik"},
187         {"Perpustakaan", "Lab Komputer"},
188         {"Perpustakaan", "Aula"},
189         {"Fakultas Teknik", "Kantin"},
190         {"Lapangan", "Masjid"},
191         {"Masjid", "Kantin"}
192     };
193     for (String[] d_1009 : diag_1009) {
194         String mark_1009 = isConsecutive_1009(d_1009[0], d_1009[1]) ? " <-- on path" : "";
195         sb_1009.append(" ").append(d_1009[0]).append(" ~ ").append(d_1009[1]).append(mark_1009).append("\n");
196     }
197     sb_1009.append("\nLegenda: |x|=belum dikunjungi >x<=dikunjungi *x*=jalur hasil");
198     graphArea_1009.setText(sb_1009.toString());
199 }

```

```

201 private String padCenter_1009(String s_1009, int width_1009) {
202     if (s_1009.length() >= width_1009) return s_1009.substring(0, width_1009);
203     int pad_1009 = width_1009 - s_1009.length(); left_1009 = pad_1009 / 2;
204     return " ".repeat(left_1009) + s_1009 + " ".repeat(pad_1009 - left_1009);
205 }
206
207 private String nameAt_1009(int row_1009, int col_1009) {
208     for (int i_1009 = 0; i_1009 < NODES_1009.length; i_1009++)
209         if (GRID_POS_1009[i_1009][1] == row_1009 && GRID_POS_1009[i_1009][0] == col_1009)
210             return NODES_1009[i_1009];
211     return null;
212 }
213
214 private boolean isConsecutive_1009(String a_1009, String b_1009) {
215     for (int i_1009 = 0; i_1009 < foundPath_1009.size() - 1; i_1009++) {
216         String x_1009 = foundPath_1009.get(i_1009), y_1009 = foundPath_1009.get(i_1009 + 1);
217         if ((x_1009.equals(a_1009) && y_1009.equals(b_1009)) || (x_1009.equals(b_1009) && y_1009.equals(a_1009)))
218             return true;
219     }
220     return false;
221 }
222
223 private Set<String> edgeSet_1009(String... pairs_1009) {
224     Set<String> s_1009 = new HashSet<>();
225     for (int i_1009 = 0; i_1009 < pairs_1009.length; i_1009 += 2)
226         s_1009.add(key_1009(pairs_1009[i_1009], pairs_1009[i_1009 + 1]));
227     return s_1009;
228 }
229
230 private boolean hasEdge_1009(Set<String> set_1009, String a_1009, String b_1009) {
231     return a_1009 != null && b_1009 != null && set_1009.contains(key_1009(a_1009, b_1009));
232 }
233
234 private String key_1009(String a_1009, String b_1009) {
235     return a_1009.compareTo(b_1009) < 0 ? a_1009 + "|" + b_1009 : b_1009 + "|" + a_1009;
236 }
237
238 public void displayPath(String algo_1009, String start_1009, String goal_1009) {
239     StringBuilder sb_1009 = new StringBuilder();
240     sb_1009.append("Algoritma : ").append(algo_1009).append("\n");
241     sb_1009.append("Start : ").append(start_1009).append("\n");
242     sb_1009.append("Goal : ").append(goal_1009).append("\n");
243     sb_1009.append("Urutan dikunjungi : ").append(visitedOrder_1009.size()).append(" node:\n");
244     sb_1009.append(String.join(" ", visitedOrder_1009)).append("\n");
245     if (foundPath_1009.isEmpty()) {
246         sb_1009.append("Jalur tidak ditemukan.");
247     } else {
248         sb_1009.append("Jalur ditemukan : ").append(foundPath_1009.size()).append(" node:\n");
249         sb_1009.append(String.join(" ", foundPath_1009));
250     }
251     resultArea_1009.setText(sb_1009.toString());
252 }
253
254 public void resetGraph() {
255     foundPath_1009.clear(); visitedOrder_1009.clear(); visitedNodes_1009.clear();
256     resultArea_1009.setText("");
257     displayGraph();
258 }
259
260 private void runSearch_1009(boolean isBFS_1009) {
261     String start_1009 = (String) startBox_1009.getSelectedItem();
262     String goal_1009 = (String) goalBox_1009.getSelectedItem();
263     if (start_1009 == null || goal_1009 == null) return;
264     if (start_1009.equals(goal_1009)) { resultArea_1009.setText("Start dan Goal tidak boleh sama."); return; }
265     String algo_1009 = isBFS_1009 ? "BFS" : "DFS";
266     foundPath_1009 = isBFS_1009 ? BFS(start_1009, goal_1009) : DFS(start_1009, goal_1009);
267     visitedNodes_1009 = new LinkedHashSet<>(visitedOrder_1009);
268     displayGraph();
269     displayPath(algo_1009, start_1009, goal_1009);
270 }
271
272 private void buildUI_1009() {
273     setTitle("BFS & DFS - Peta Kampus | 2511531009");
274     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
275     setLayout(new BorderLayout(8, 8));
276
277     JPanel ctrl_1009 = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 6));
278     String[] nodes_1009 = graph_1009.keySet().toArray(new String[0]);
279     startBox_1009 = new JComboBox<>(nodes_1009);
280     goalBox_1009 = new JComboBox<>(nodes_1009);
281     goalBox_1009.setSelectedIndex(nodes_1009.length - 1);
282
283     JButton bfsBtn_1009 = new JButton("BFS");
284     JButton dfsBtn_1009 = new JButton("DFS");
285     JButton resetBtn_1009 = new JButton("Reset");
286     ctrl_1009.add(new JLabel("Start:")); ctrl_1009.add(startBox_1009);
287     ctrl_1009.add(new JLabel("Goal:")); ctrl_1009.add(goalBox_1009);
288     ctrl_1009.add(bfsBtn_1009); ctrl_1009.add(dfsBtn_1009); ctrl_1009.add(resetBtn_1009);
289     add(ctrl_1009, BorderLayout.NORTH);
290
291     graphArea_1009 = new JTextArea(12, 80);
292     graphArea_1009.setEditable(false);
293     graphArea_1009.setFont(new Font("Monospaced", Font.PLAIN, 13));
294     add(new JScrollPane(graphArea_1009), BorderLayout.CENTER);
295
296     resultArea_1009 = new JTextArea(7, 80);
297     resultArea_1009.setEditable(false);
298     resultArea_1009.setFont(new Font("Monospaced", Font.PLAIN, 13));
299     add(new JScrollPane(resultArea_1009), BorderLayout.SOUTH);
300
301     bfsBtn_1009.addActionListener(e_1009 -> runSearch_1009(true));
302     dfsBtn_1009.addActionListener(e_1009 -> runSearch_1009(false));
303     resetBtn_1009.addActionListener(e_1009 -> resetGraph());
304
305     pack();
306     setLocationRelativeTo(null);
307     displayGraph();
308 }
309
310 public static void main(String[] args_1009) {
311     SwingUtilities.invokeLater(() -> new GraphSearchGUI_2511531009().setVisible(true));
312 }

```

BFS & DFS - Peta Kampus | 2511531009

Start: Goal:

```

+-----+ +-----+ +-----+
* Fakultas Teknik ***** Lab Komputer *---* Aula *
+-----+ +-----+ +-----+
| | |
+-----+ +-----+ +-----+
* Gerbang Utama ***** Gedung Rektorat *---* Perpustakaan *---| Kantin |---* Lapangan *
+-----+ +-----+ +-----+
| | |
+-----+ +-----+
| Parkiran | | Masjid |
+-----+ +-----+

```

(~) diagonal/long edges:
 Gedung Rektorat ~ Fakultas Teknik <-- on path
 Perpustakaan ~ Lab Komputer <-- on path
 Perpustakaan ~ Aula <-- on path
 Fakultas Teknik ~ Kantin
 Lapangan ~ Masjid
 Masjid ~ Kantin

Legenda: |x|=belum dikunjungi >x<=dikunjungi *x*=jalur hasil

Algoritma : DFS
 Start : Gerbang Utama
 Goal : Lapangan

Urutan dikunjungi (7 node):
 Gerbang Utama → Gedung Rektorat → Fakultas Teknik → Lab Komputer → Perpustakaan → Aula → Lapangan

BFS & DFS - Peta Kampus | 2511531009

Start: Goal:

```

+-----+ +-----+ +-----+
> Fakultas Teknik <---> Lab Komputer <---> Aula <
+-----+ +-----+ +-----+
| | |
+-----+ +-----+ +-----+
* Gerbang Utama ***** Gedung Rektorat *---> Perpustakaan <---> Kantin <---* Lapangan *
+-----+ +-----+ +-----+
| | |
+-----+ +-----+
> Parkiran < * Masjid *
+-----+ +-----+

```

(~) diagonal/long edges:
 Gedung Rektorat ~ Fakultas Teknik
 Perpustakaan ~ Lab Komputer
 Perpustakaan ~ Aula
 Fakultas Teknik ~ Kantin
 Lapangan ~ Masjid <-- on path
 Masjid ~ Kantin

Legenda: |x|=belum dikunjungi >x<=dikunjungi *x*=jalur hasil

Algoritma : BFS
 Start : Gerbang Utama
 Goal : Lapangan

Urutan dikunjungi (10 node):
 Gerbang Utama → Gedung Rektorat → Parkiran → Fakultas Teknik → Perpustakaan → Masjid → Lab Komputer → Kantin